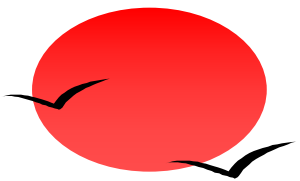


软件设计师

案例分析专题 试题3

UML建模

讲师：诸葛老师



1 考点分析

- 根据考试大纲，**UML建模**已经成为案例分析的固定试题三，占**15分**，主要考查UML建模过程中的一些重要图形，如**类图**、**用例图**、状态图、顺序图、活动图等。

- **1. 类图**

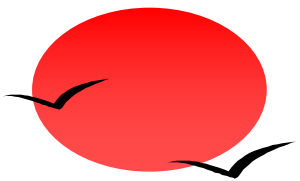
主要考查**补充类名**、**类的主要属性和方法**、**类之间的多重度**、**类之间的关系**（泛化、关联、依赖、聚合、组合、实现）。

- **2. 用例图**

主要考查**补充参与者名**、**用例名**、**用例之间的关系**（包含include、扩展extend、泛化）等。

- **3. 顺序图**

主要考查**补充对象名**、**消息名**以及对象间消息传递的时间顺序。



1 考点分析

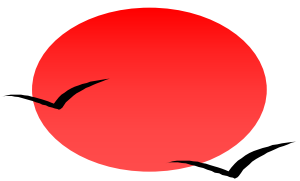
■ 4. 活动图

主要考查补充活动结点的名称、各活动之间的关系。

■ 5. 状态图

主要考查补充状态名、状态转换的条件。

软考诸葛老师

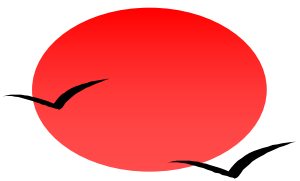


2 UML建模

- UML中有四种关系：**依赖**、**关联**、**泛化和实现**，如下表所示。

名称	子集	解释	举例	图形
关联	关联	两个类之间存在某种语义上的联系，执行者与用例的关系（描述了一组链，链是对象之间的连接）	人和公司有某种关联	
	聚合	整体与部分的关系。（部分离开整体可独立存在）	狼与狼群的关系	
	组合	整体与部分的关系。（部分离开整体不可独立存在）	车轮与车的关系	
泛化		一种特殊/一般关系，父类和子类之间的关系。 一般事务与该事务中特殊种类之间的关系	人与老师的继承关系	
实现		规定接口和实现接口的类或组件之间的关系		
依赖		有包含、扩展等关系	人依赖食物	

- 依赖关系最弱，只要有部分相关就是依赖。泛化关系是父子关系；组合和聚合都是部分-整体关系，组合更强一些。



2 UML建模

■ 1. 类图

描述类、类的特性以及类之间的关系。

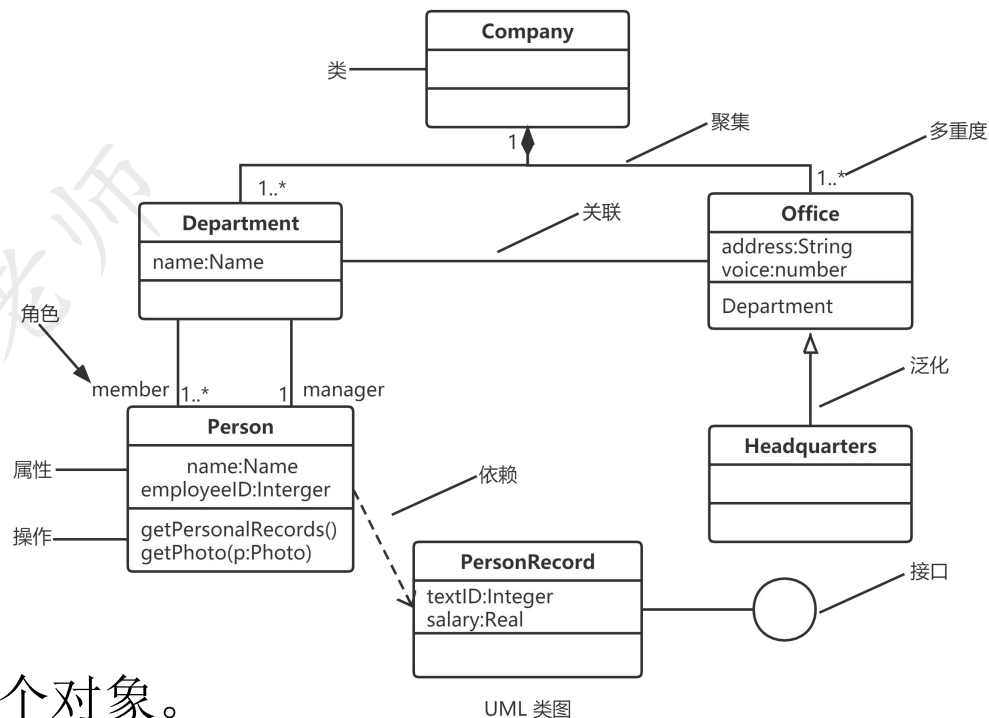
多重度含义：

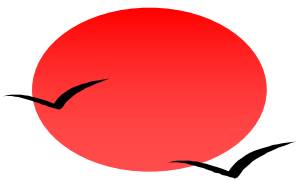
1：表示一个集合中的1个对象对应另一个集合中的1个对象。

0..*：表示一个集合中的1个对象对应另一个集合中的0个或多个对象。

1..*：表示一个集合中的1个对象对应另一个集合中的1个或多个对象。

*：表示一个集合中的1个对象对应另一个集合中的多个对象。





2 UML建模

■ 2. 用例图

描述了一组用例、参与者以及它们之间的关系。

参与者一般是人、硬件或其他系统可以扮演的角色；

用例是参与者完成的一系列操作。

用例之间的关系有3种：

①包含关系。当可以从两个或两个以上的用例中提取公共行为时，应该使用包含关系来表示它们，用<<include>>表示。

②扩展关系。如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，这样的描述可能更加清晰，用<<extend>>表示。

③泛化关系。当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。

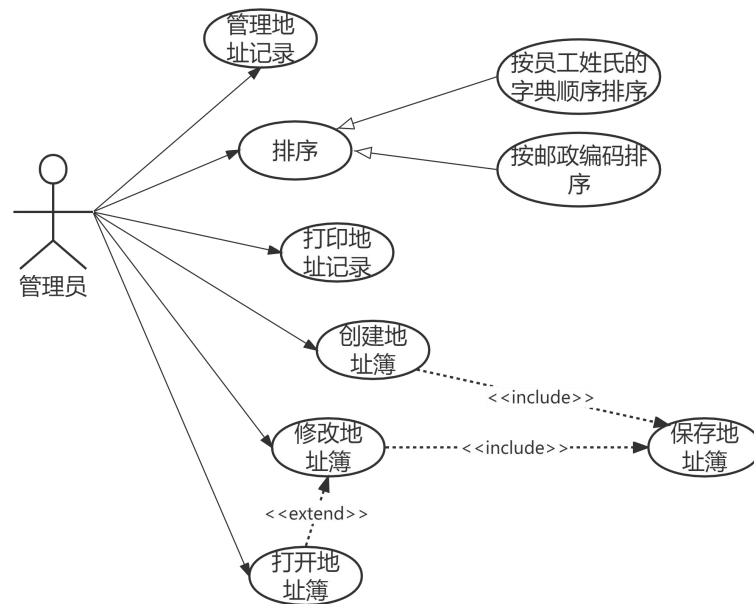
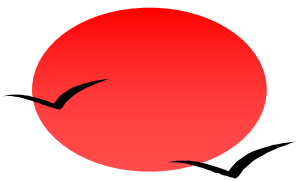


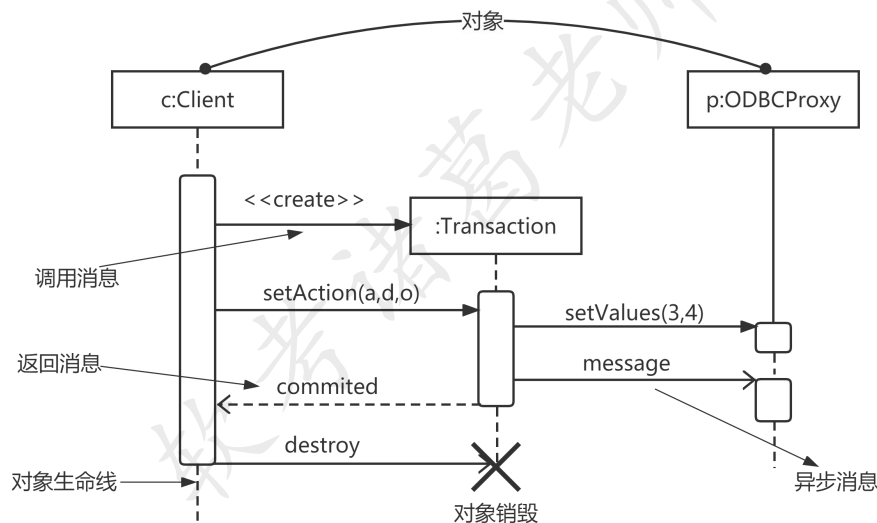
图3-1 用例图



2 UML建模

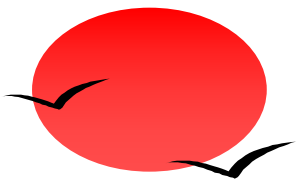
■ 3. 顺序图

又称**序列图**，描述**对象之间的交互**（消息的发送与接收），重点在于**强调顺序**，反映对象间消息的发送与接收。



UML顺序图

数据库使用数据库代理的顺序图：在顺序图中，首先把参加交互的对象放在图的最上方，沿水平方向排列。通常把发起交互的对象放在左边，下级对象依次放在右边。然后把这些对象发送和接收的消息沿垂直方向按时间顺序从上到下放置。这样，就提供了控制流随时间推移的清晰的可视化轨迹。（消息在箭头上传递，对象作为实体在最上端）

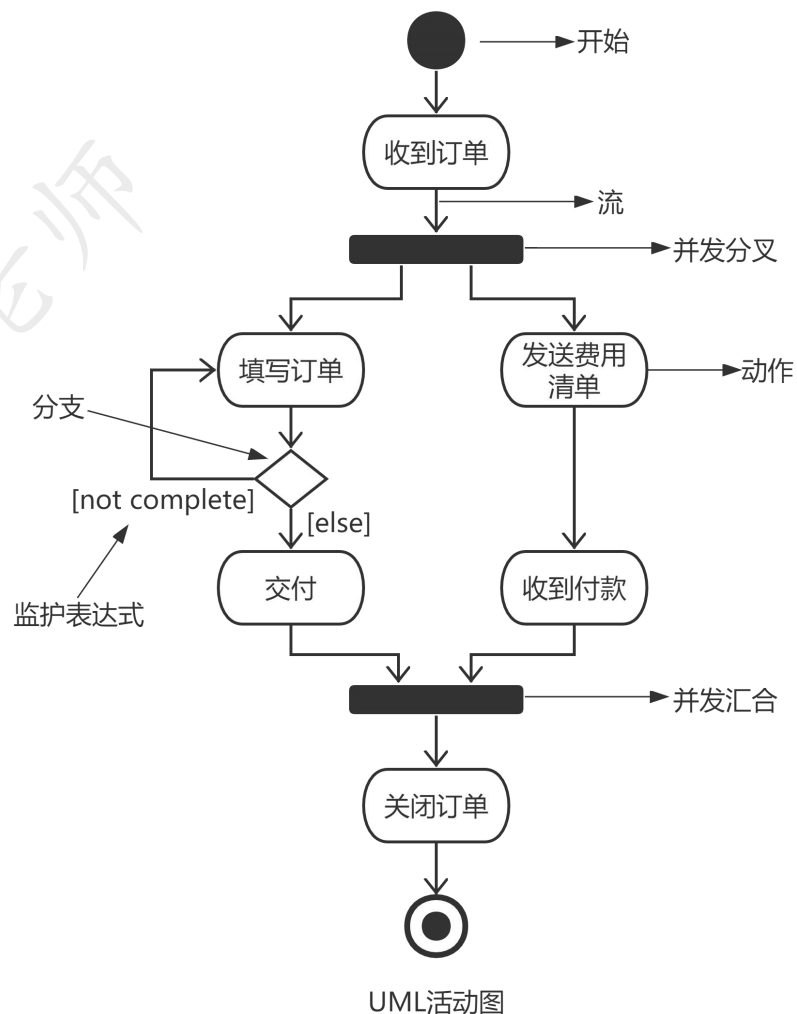


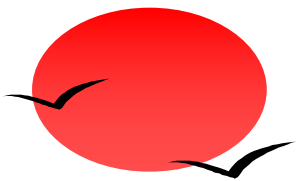
2 UML建模

■ 4. 活动图

描述过程行为和并行行为。

它是一种特殊的状态图，展现了在系统内从一个活动到另一个活动的流程，对于系统的功能建模特别重要，并强调对象间的控制流程。





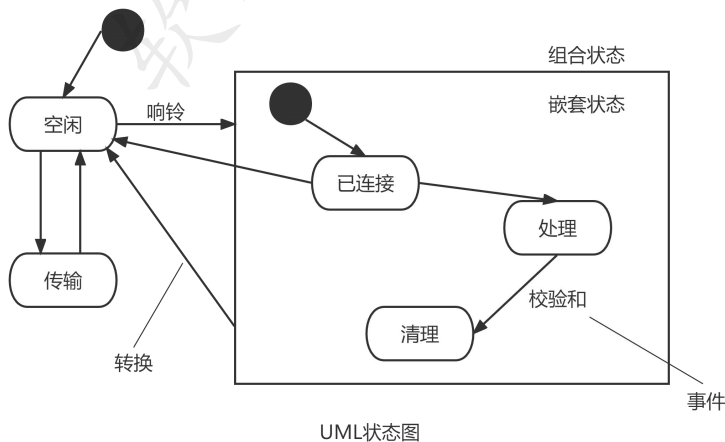
2 UML建模

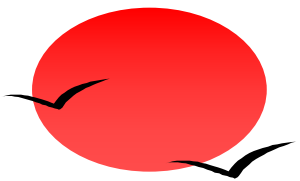
■ 5. 状态图

描述对象状态及其转换。

就是一个状态机，由状态、转换、事件等组成。状态是指对象在其生命周期中的某个条件或状态。转换可以通过事件触发，事件触发后相应的监护条件会进行检查。如下图中圆角矩形表示状态，箭头上表示事件，实心圆表示起点和终点。状态图关注系统的动态视图，强调对象行为的事件顺序。

这是一个接电话的状态图，状态图的状态通常包括简单状态和组合状态（含有子状态的状态称为组合状态），除此之外状态图中还有转换、事件等。





3 解题技巧

- 由前面的介绍可知，考查UML建模就是考查多种图形，对这些图形的考查一般都是缺失一些关键点，要求考生补图。

认真审题审图，根据说明查漏补缺，一般来说有以下几种题型：

- (1) 补充类图（常考）

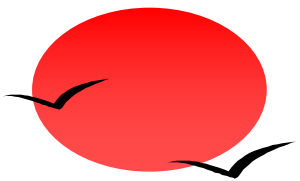
主要考查补充类名称，需要根据类之间的关系以及多重度来判断，需要牢记类之间关系的图形符号，尤其是组合、聚合和继承的符号，并且观察符号上的多重度数字，与题目描述对应。

- (2) 补充用例图（常考）

主要考查补充用例、参与者以及用例之间的关系，只要认真审题，根据题中描述核对，都可以轻易得出答案。

- (3) 补充顺序图、活动图、状态图

认识图的基本元素，仔细审题，仔细阅读题目描述。



本节练习-UML建模

- 阅读下列说明和图，回答问题1至问题3，将解答填入答题纸的对应栏内。

【说明】

图3-1所示为某软件系统中一个温度控制模块的界面。界面上提供了两种温度计量单位，即华氏度（Fahrenheit）和摄氏度（Celsius）。软件支持两种计量单位之间的自动换算，即若输入一个华氏度的温度，其对应的摄氏度温度值会自动出现在摄氏度的显示框内，反之亦然。

用户可以通过该界面上的按钮Raise(升高温度)和Lower(降低温度)来改变温度的值。界面右侧是个温度计，将数字形式的温度转换成温度计上的刻度比例进行显示。当温度值改变时，温度计的显示也随之同步变化。

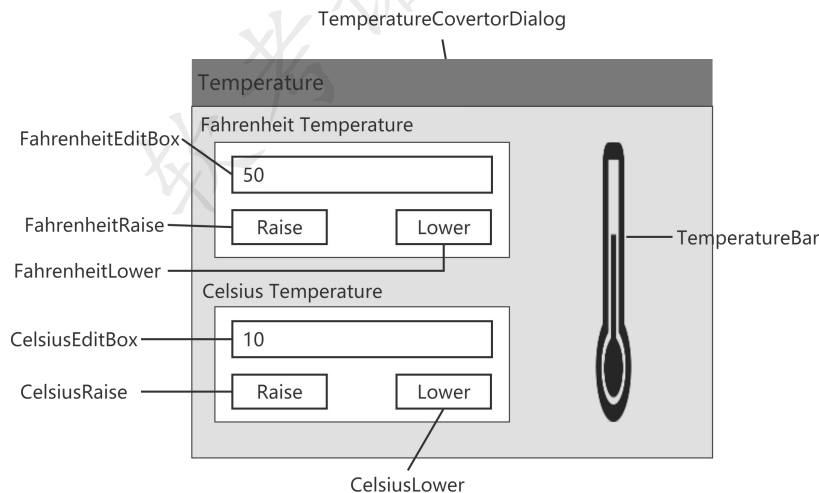
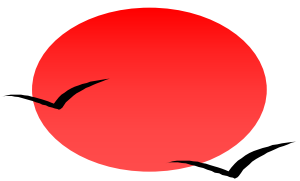


图3-1 温度控制模块的界面



本节练习-UML建模

- 现在采用面向对象方法实现该温度控制模板，得到如图3-2所示的用例图和3-3所示的类图。

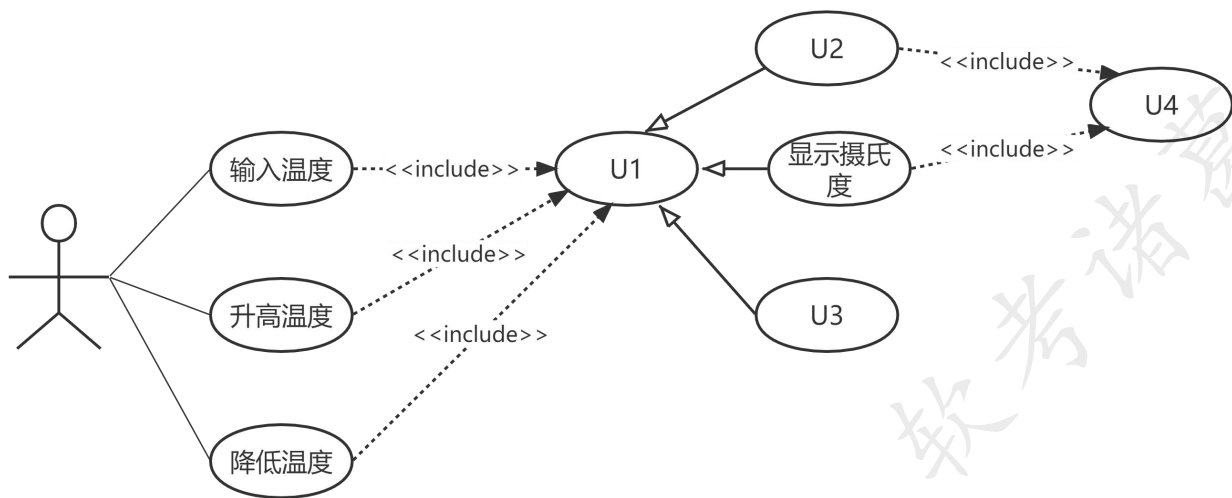


图3-2 用例图

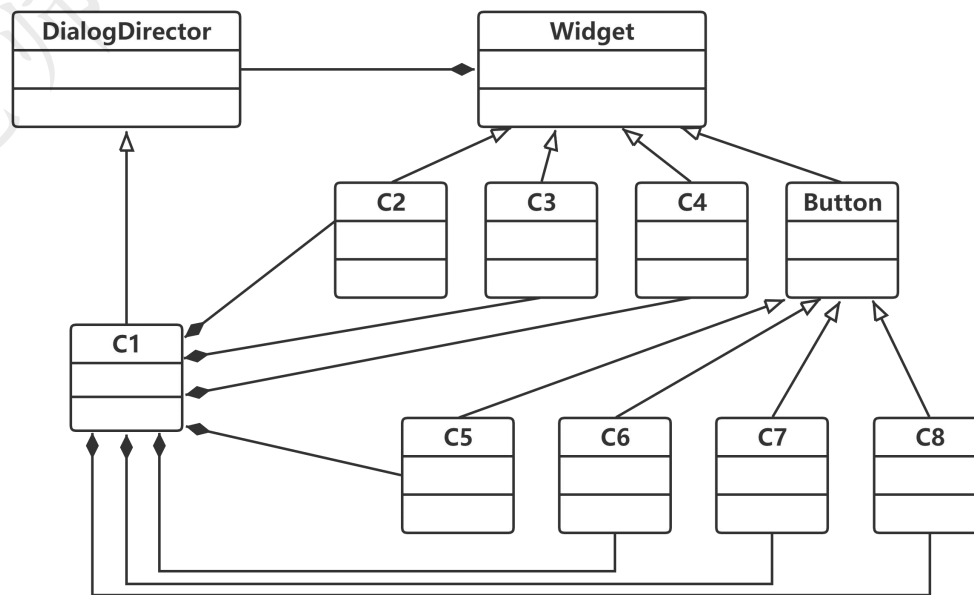
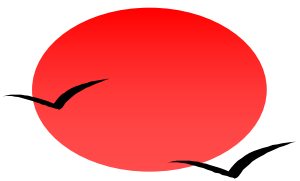


图3-3 类图



本节练习-UML建模

图3-1所示为某软件系统中一个温度控制模块的界面。界面上提供了两种温度计量单位，即华氏度(Fahrenheit)和摄氏度(Celsius)。软件支持两种计量单位之间的自动换算，即若输入一个华氏度的温度，其对应的摄氏度温度值会自动出现在摄氏度的显示框内，反之亦然。

用户可以通过该界面上的按钮Raise(升高温度)和Lower(降低温度)来改变温度的值。界面右侧是个温度计，将数字形式的温度转换成温度计上的刻度比例进行显示。当温度值改变时，温度计的显示也随之同步变化。

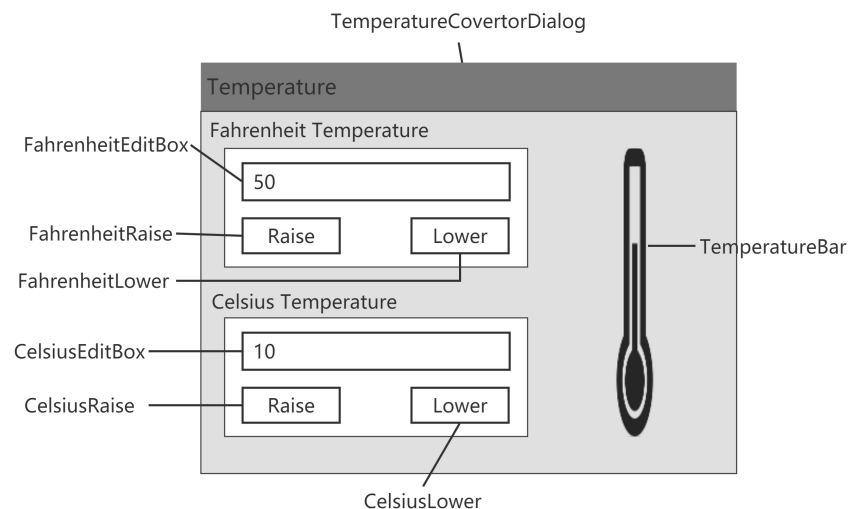


图3-1 温度控制模块的界面

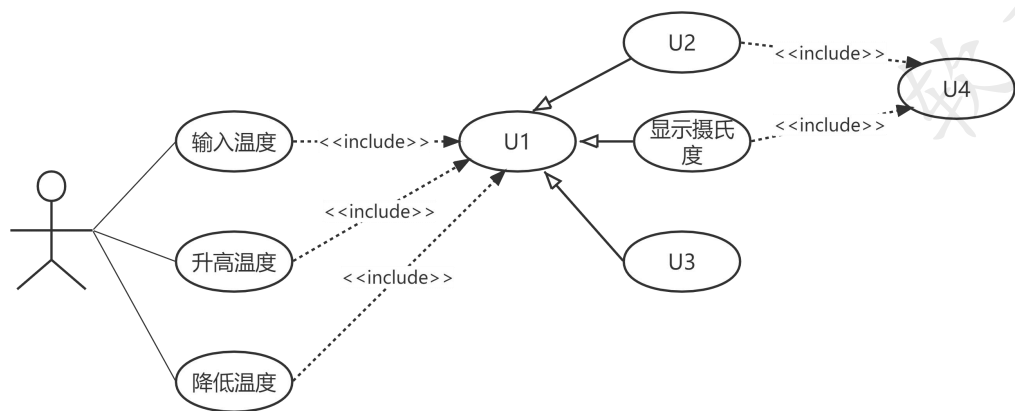


图3-2 用例图

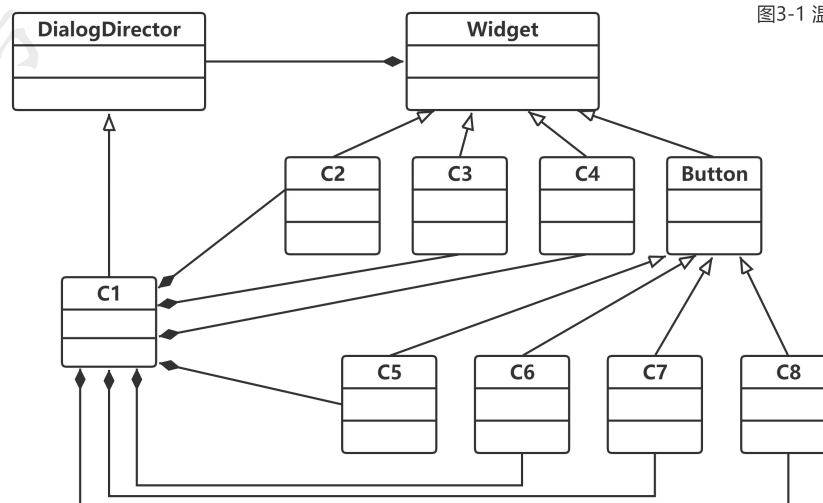
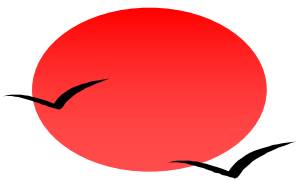


图3-3 类图



本节练习-UML建模

- 【问题1】（6分）

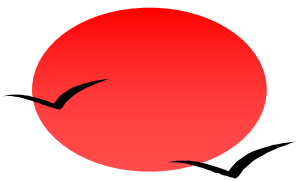
根据说明中的描述，给出图3-2中U1~U4所对应的用例名。

- 【问题2】（6分）

根据说明中的描述，给出图3-3中C1~C8所对应的类名(类名使用图3-1中标注的词汇)。

- 【问题3】（3分）

现需将图3-1所示的界面改造为一个更为通用的GUI应用，能够实现任意计量单位之间的换算，例如千克和克之间的换算、厘米和英寸之间的换算等等。为了实现这个新的需求，可以在图3-3所示的类图上增加哪种设计模式？请解释选择该设计模式的原因(不超过50字)。



本节练习-UML建模

- 【参考答案】

- 【问题1】 U2: 显示温度 U2: 显示华氏度 U3: 显示温度计 U4: 单位自动换算

- 【问题2】

C1: TemperatureCovertorDialog

C5~C8: (顺序可互换)

C2~C4: (顺序可互换)

FahrenheitRaise

FahrenheitEditBox

FahrenheitLower

CelsiusEditBox

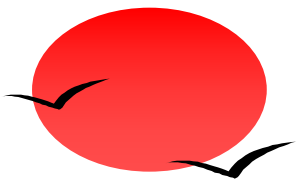
CelsiusRaise

TemperatureBar

CelsiusLower

- 【问题3】

策略模式。原因：多种单位之间不同的换算规则可以写成不同的算法策略，这些算法之间是平等关系，可以根据自己的需求选择不同的策略，并且可以灵活切换。策略模式，此模式可以定义一系列的算法策略，把它们一个个封装起来，并且使它们可以相互替换。在需要指定计量单位换算时，调用相应的算法策略即可。（类似所表达的意思即可）



本节练习-UML建模

- 阅读下列说明，回答问题1至问题3，将解答填入答题纸的对应栏内。

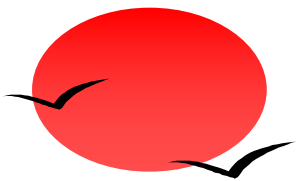
【说明】

某公司的人事部门拥有一个地址簿(AddressBook)管理系统(AddressBookSystem)，用于管理公司所有员工的地址记录(PersonAddress)。员工的地址记录包括：姓名、住址、城市、省份、邮政编码以及联系电话等信息。管理员可以完成对地址簿中地址记录的管理操作，包括：

- (1) 管理地址记录。根据公司的人员变动情况，对地址记录进行添加、修改、删除等操作。
- (2) 排序。按照员工姓氏的字典顺序或邮政编码对地址簿中的所有记录进行排序。
- (3) 打印地址记录。以邮件标签的格式打印一个地址单独的地址簿。系统会对地址记录进行管理，为便于管理，管理员在系统中为公司的不同部门建立员工的地址簿的操作，包括：

- (1) 创建地址簿。新建一个地址簿并保存。
- (2) 打开地址簿。打开一个已有的地址簿。
- (3) 修改地址簿。对打开的地址簿进行修改并保存。

系统将提供一个GUI（图形用户界面）实现对地址簿的各种操作。



本节练习-UML建模

- 现采用面向对象方法分析并设计该地址簿管理系统，得到如图3-1所示的用例图和图3-2所示的类图。

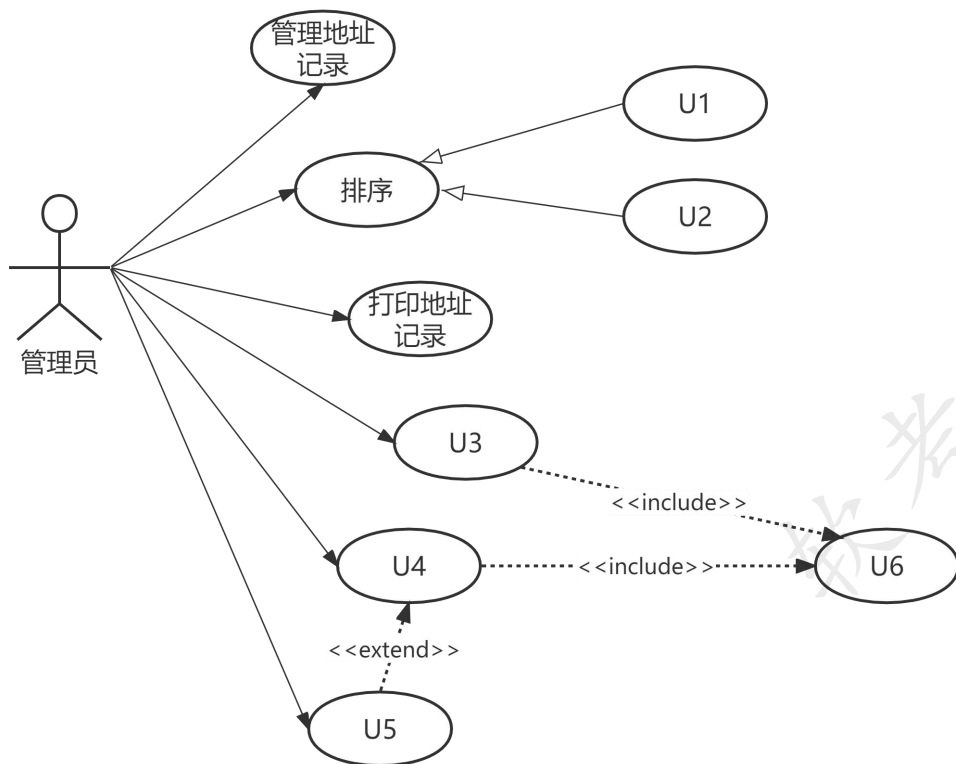


图3-1 用例图

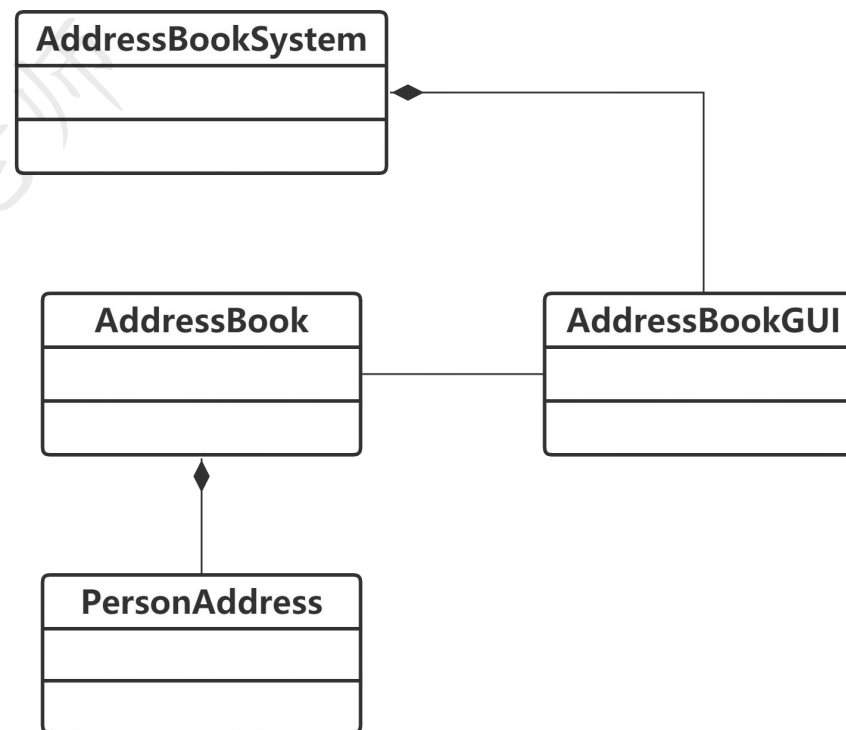
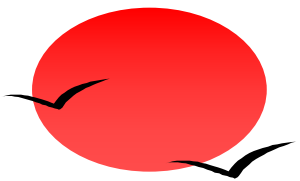


图3-2 类图



本节练习-UML建模

- 某公司的人事部门拥有一个地址簿(AddressBook)管理系统(AddressBookSystem)，用于管理公司所有员工的地址记录(PersonAddress)。员工的地址记录包括：姓名、住址、城市、省份、邮政编码以及联系电话等信息。管理员可以完成对地址簿中地址记录的管理操作，包括：
 - (1) 管理地址记录。根据公司的人员变动情况，对地址记录进行添加、修改、删除等操作。
 - (2) 排序。按照员工姓氏的字典顺序或邮政编码对地址簿中的所有记录进行排序。
 - (3) 打印地址记录。以邮件标签的格式打印一个地址单独的地址簿。系统会对地址记录进行管理，为便于管理，管理员在系统中为公司的不同部门建立员工的地址簿的操作，包括：
 - (1) 创建地址簿。新建一个地址簿并保存。
 - (2) 打开地址簿。打开一个已有的地址簿。
 - (3) 修改地址簿。对打开的地址簿进行修改并保存。

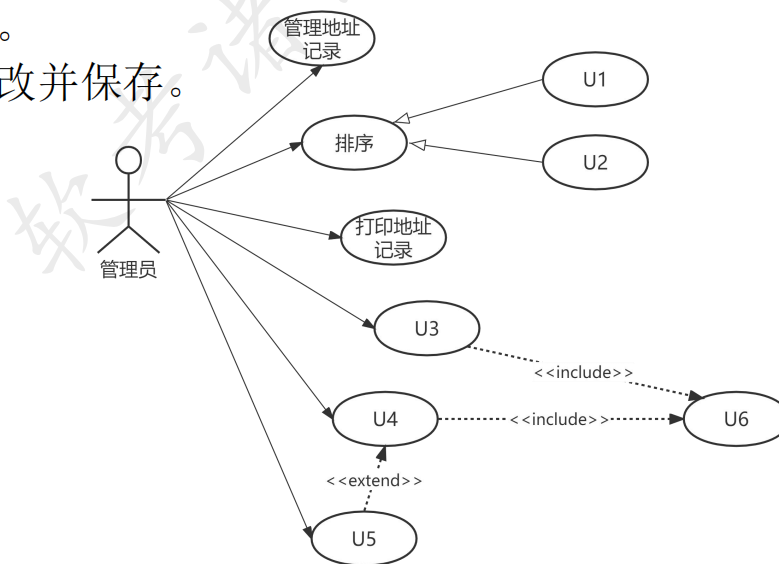


图3-1 用例图

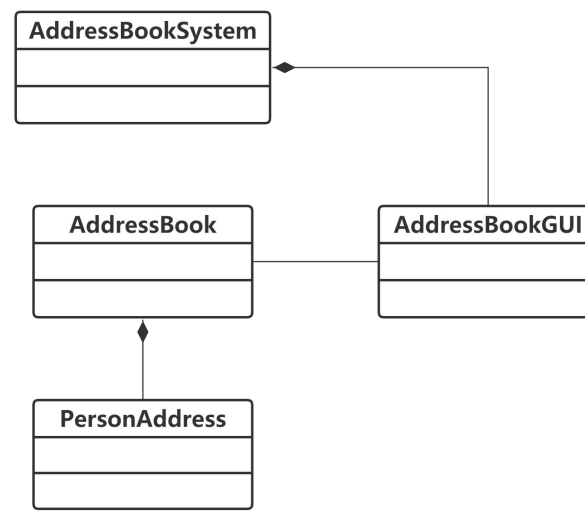
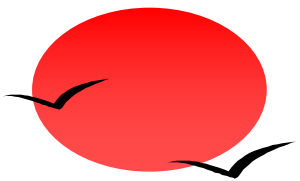


图3-2 类图



本节练习-UML建模

- 【问题1】（6分）

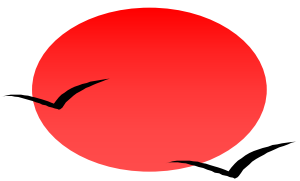
根据说明中的描述，给出图3-1中U1~U6所对应的用例名。

- 【问题2】（5分）

根据说明中的描述，给出图3-2中类AddressBook的主要属性和方法以及类PersonAddress的主要属性（可以使用说明中的文字）。

- 【问题3】（4分）

根据说明中的描述以及图3-1所示的用例图，请简要说明include和extend关系的含义是什么？



本节练习-UML建模

- 【参考答案】

- 【问题1】

U1: 按员工姓氏的字典顺序排序

U2: 按邮政编码排序（U1和U2可互换）

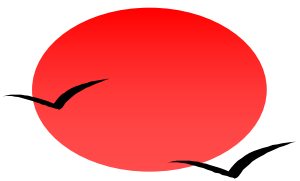
U3: 创建地址簿

U4: 修改地址簿

U5: 打开地址簿

U6: 保存地址簿

软考诸葛老师



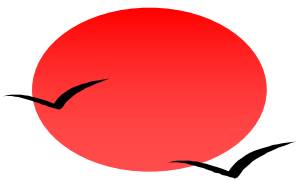
本节练习-UML建模

■ 【问题2】

类PersonAddress也就是员工地址记录，根据题干描述“员工的地址记录包括姓名、住址、城市、省份、邮政编码以及联系电话等信息”，因此类PersonAddress的主要属性包括姓名、住址、城市、省份、邮政编码以及联系电话等。

类AddressBook也就是地址簿用于管理公司所有员工的地址记录，由于“管理员在系统中为公司的不同部门建立员工的地址簿”，因此类AddressBook的主要属性除了员工地址记录信息以外，还需要增加部门信息。此时类AddressBook的主要属性包括：部门名/编号，姓名、住址、城市、省份、邮政编码以及联系电话等。

“管理地址记录，根据公司的人员变动情况，对地址记录进行添加、修改、删除等操作”，因此类AddressBook的方法需要包括添加、修改、删除地址记录等管理地址记录操作。又因为“管理员在系统中为公司的不同部门建立员工的地址簿的操作，包括（1）创建地址簿。新建一个地址簿并保存。（2）打开地址簿。打开一个已有的地址簿。（3）修改地址簿。对打开的地址簿进行修改并保存。”，因此类AddressBook的方法需要包括创建地址簿、打开地址簿、修改地址簿、保存地址簿。综上，类AddressBook的主要方法包括管理地址记录、排序地址记录、打印地址记录、创建地址簿、打开地址簿、修改地址簿、保存地址簿₂₁

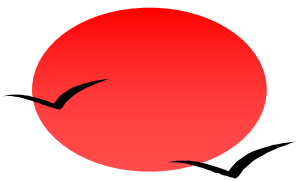


本节练习-UML建模

■ 【问题3】

`include`属于用例图的三种关系之一，表示的是**包含关系**。描述为当可以从两个或两个以上用例中提取公共行为的时候，应该使用包含关系来表示它们。其中这个提取出来的公共用例称之为抽象用例，而把原始用例称为基本用例和扩展用例。

`extend`属于用例图的三种关系之一，表示的是**扩展关系**。描述为如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能会发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，关系图示指向基本用例。



感谢

THANKS

软件设计师QQ群: 759713504

诸葛老师微信: zhugeruankao